24

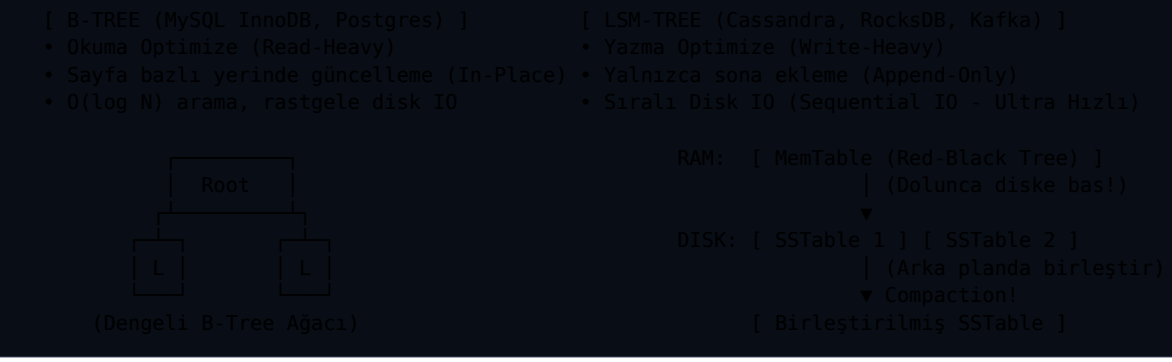
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Veritabanı İndeksleme Sırları: B-Tree vs LSM-Tree (Log-Structured Merge-Tree)

Database Indexing Under the Hood: B-Tree vs LSM-Tree

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

B-Tree, kalın ve çok düzenli bir kütüphane indeks fihristi gibidir: Yeni bir kitap geldiğinde fihristin ilgili sayfasına gidip araya sıkıştırarak yazarsınız (Yerinde Güncelleme - In-Place Update). Okurken harikadır, aradığınızı saniyesinde bulursunuz; ama çok fazla yeni kitap gelirse sayfaları silip yeniden düzenlemekten kütüphanecinin eli yorulur. LSM-Tree ise seri bir günlük yazarı gibidir: Asla eski sayfaları silmez! Ne olursa olsun günlüğün en sonuna yazar (Append-Only). Yazmak ışık hızındadır! Akşam olunca da arkada oturup günlükteki tekrarları temize çeker (Compaction)!

🇬🇧 ENGLISH (ELI5)

A B-Tree is a meticulously maintained card catalog in a library: When a new book arrives, the librarian navigates to the exact drawer and slots the card into alphabetical position (In-Place mutation). Reading is instantaneous ($O(log N)$), but heavy writes force costly random disk seeks. An LSM-Tree (Log-Structured Merge-Tree) is a court stenographer typing at 200 words/minute: They NEVER erase or rewrite past pages; they append every new event strictly to the end of the roll (Sequential Disk IO). Later, background janitors compress and deduplicate old logs into sorted tables (Compaction)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) B-Tree / B+Tree: İlişkisel veritabanlarının (MySQL InnoDB, PostgreSQL) kalbidir. Veri 16 KB'lık sayfalarda (Pages) dengeli bir ağaç yapısında tutulur. Sayfalar doğrudan yerinde (In-Place) güncellenir. Rastgele disk IO gerektirdiği için yazma maliyeti yüksektir; ancak okumalar tek bir sayfa aramasıyla tamamlanır ($O(log N)$). 2) LSM-Tree: Modern NoSQL ve depolama motorlarının (RocksDB, Cassandra, LevelDB) kalbidir. Yazmalar önce RAM'deki sıralı bir ağaca (`MemTable`) ve kurtarma günlüğüne (`WAL`) yazılır (sıfır rastgele disk IO!). MemTable dolunca diske sıralı ve değiştirilemez (Immutable) `SSTable` (Sorted String Table) olarak dökülür. Okuma yaparken: MemTable -> Bloom Filter -> SSTable dosyaları taranır. Arka planda `Compaction` işlemi eski sürümleri temizler.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) B-Tree / B+Tree: The classic relational storage engine (Postgres, InnoDB). Organizes fixed-size pages (typically 16KB) in a balanced tree. Updates perform random in-place page writes. Read paths are lightning fast ($O(log N)$ disk seeks), but high-velocity random writes fragment storage and cause heavy write amplification. 2) LSM-Tree (Log-Structured Merge-Tree): Powers modern high-throughput NoSQL engines (Cassandra, RocksDB, ClickHouse). Writes append sequentially to an in-memory sorted tree (`MemTable`) and a durable WAL, converting random writes into maximum-speed sequential IO. Saturated MemTables flush immutably to disk as `SSTables`. Background merge-compaction processes consolidate fragmented segments and evict deleted tombstones.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: LSM-Tree MemTable & SSTable Döküm Mantığı Simülasyonu
import java.util.*;

public class LsmTreeSim {
    // RAM'deki Sıralı Bellek Tablosu (MemTable)
    private final NavigableMap<String, String> memTable = new TreeMap<>();
    private final List<Map<String, String>> ssTablesOnDisk = new ArrayList<>();
    private static final int MEMTABLE_THRESHOLD = 3;

    public synchronized void put(String key, String value) {
        memTable.put(key, value); // RAM'e sıralı yaz (O(log N))
        if (memTable.size() >= MEMTABLE_THRESHOLD) {
            flushToSsTable();
        }
    }

    private void flushToSsTable() {
        // Diske sıralı, değiştirilemez SSTable olarak dök:
        ssTablesOnDisk.add(new HashMap<>(memTable));
        memTable.clear();
        IO.println(">>> MemTable Doldu! Diske SSTable döküldü (Sequential Disk IO). Toplam SSTable: "
+ ssTablesOnDisk.size());
    }

    public synchronized String get(String key) {
        // 1. Önce RAM'deki MemTable'a bak:
        if (memTable.containsKey(key)) return memTable.get(key);
        // 2. Bulunamazsa en yeni SSTable'dan eskiye doğru diski tara:
        for (int i = ssTablesOnDisk.size() - 1; i >= 0; i--) {
            if (ssTablesOnDisk.get(i).containsKey(key)) return ssTablesOnDisk.get(i).get(key);
        }
        return null;
    }
}

void main() {
    var lsm = new LsmTreeSim();
    lsm.put("user_1", "Ali");
    lsm.put("user_2", "Bob");
    lsm.put("user_3", "Can"); // Eşik doldu -> SSTable diske basıldı!
    lsm.put("user_4", "Deniz");

    IO.println("Sorgu 'user_1': " + lsm.get("user_1"));
    IO.println("Sorgu 'user_4': " + lsm.get("user_4"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

LSM-Tree'lerin zayıf karnı 'Read Amplification' (Okuma Büyümesi) ve 'Compaction' sırasında yaşanan disk IO darboğazlarıdır. Mülakatta bu dezavantajı gidermek için SSTable dosyalarının önüne mutlaka Bloom Filter konulduğunu belirtin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

LSM-Trees suffer from read amplification (checking multiple SSTable files on misses) and compaction IO storms. Explicitly state that pairing Bloom Filters with SSTables allows skipping unnecessary disk reads in 99% of queries.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• B-Tree okuma ağırlıklı sistemler (RDBMS) için, LSM-Tree ise yazma canavarları (NoSQL) içindir.

• LSM-Tree sıralı disk yazması (Sequential IO) kullanarak mekanik ve SSD diskleri son hızda sürer.

• Bloom Filter ve Compaction, LSM-Tree mimarisinin okuma performansını koruyan iki emniyet sibobudur.

🎯 Key Takeaways for the Interview

• B-Trees power read-heavy RDBMS; LSM-Trees dominate extreme write-heavy workloads.

• LSM-Trees convert random writes into blazing sequential append IO.

• Bloom Filters and background Compaction keep LSM-Tree read latencies tightly bounded.